

Moore Machine

As we already covered the topics of DFA and NFA. DFA and NFA are the automata machines that take the input and do not give the output. Both DFA and NFA just check the acceptance of the given string. However, the Automata Moore Machine and Mealy also take the input and give the output. There is no final state in the Moore Machine.

Condition: For the Moore machine, there must be a dedicated path for each input from each state, like DFA.

Moore and Mealy machines take an input string (i.e., 11001) and give an output (i.e., aabbab).

- Each state of the Moore or Mealy machine gives an output. Output is denoted by the Output symbol.
- **The initial state always gives an output.** The remaining outputs depend on the transition function.

Important: The output of the Moore machine is always one greater than the input. If the input length is three (i.e., 011), the output length will be four (i.e., aabb). Because the initial state always gives an output without an input.

Formal Definition of Moore Machine

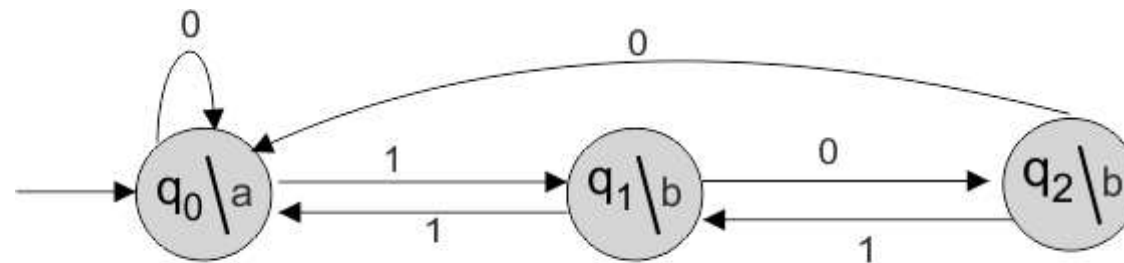
Moore machines accept all regular languages. Their output depends only on the machine's current state, which is represented by the current state separated by “/.”

Moore machine can be described by 6 tuples $(Q, q_0, \Sigma, O, \delta, \lambda)$ where,

- Q : a finite set of states
- q_0 : initial state of machine
- Σ : a finite set of input symbols
- O : output alphabet
- δ : transition function where $Q \times \Sigma \rightarrow Q$
- λ : output function where $Q \rightarrow O$

Moore Machine Example

Consider the following Moore Machine



Moore Machine

Explanation of Moore Machine Tuples

Explanation of 6 tuples $(Q, q_0, \Sigma, \delta, O, \lambda)$ is given below

1. q_0, q_1, q_2 are the states of the given Moore machine.

2. q_0 is the initial state.
 3. 0,1 are the inputs (Σ) values for transition.
 4. The transition function is performed on input and gives a state move.
 5. O is the output, where a and b are output symbols that are separated by “/” from the state symbol.
 6. Output function (λ) gives the output. As in the above Moore Machine.
 - Initial state q_0 gives output “a”.
- - If state q_0 and input “1” are given, the state is changed to q_1 , and q_1 will give the “b” output.
 - In the same way, if state q_1 and input “1” are given, then the state changes to q_2 , and q_2 will give the “b” output.
 - - Similarly, if state q_2 and input “0” are given, the state changes to q_0 , and q_0 will give the “a” output. And so on...

Input to Output in Moore Machine

For instance, if the input “101001” is given to the above Moore machine, the output will be “abbbbab”. First, “a” of output comes without input. That’s why output is always greater than input length by 1. The remaining output comes after consuming each input. As shown in the following table.

Input		1	0	1	0	0	1
State	q_0	q_1	q_2	q_1	q_2	q_0	q_1
Output	a	b	b	b	b	a	b

Moore Transition Table

As q0 gives output “a”, q1 gives output “b”, and q2 gives output “b”. Transition for each input is also covered in the DFA and NFA table. So, the Transition table for given Moore Machine is given below.

States	Output	Input “0”	Input “1”
q0	a	q0	q1
q1	b	q2	q0
q2	b	q0	q1

Moore Machine Example 01: 1’s Complement

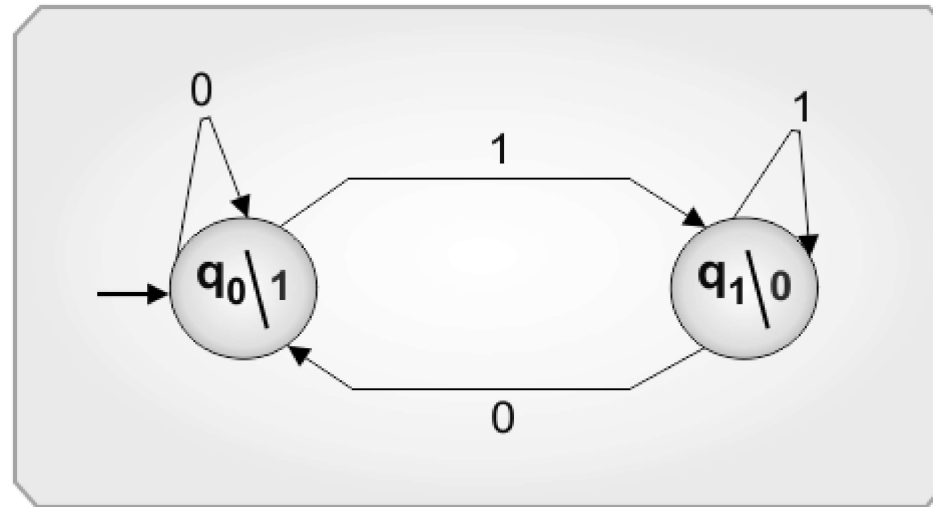
Let’s design a Moore Machine that provides the first complement of any binary input. The 1’s complement operation means reversing every bit in a binary number, changing all 0’s to 1’s and all 1’s to 0’s. For example: If the input is 1010, then the output (1’s complement) is 0101.

Design of Moore Machine

To design a Moore machine for generating the 1’s complement, we use two states:

State Output		Meaning
q0	1	Produces output 1 (used when input is 0)
q1	0	Produces output 0 (used when input is 1)

Moore Transition Diagram



First Complement of Moore Machine

Moore Transition Table

Present State	Input = 0	Input = 1	Output
q_0	q_0	q_1	1
q_1	q_0	q_1	0

Explanation

- When the input bit is 0, the machine moves to q_0 , and the output is 1.
- When the input bit is 1, the machine moves to q_1 , and the output is 0.

The machine changes states according to the input bit and gives the 1's complement as output.

Moore Machine Example Execution

Let the input be 1 0 1 1.

Input Bit	Next State	Output
1	q1	0
0	q0	1
1	q1	0
1	q1	0

Final Output: 0 1 0 0

Moore Machine Example 02: 2's Complement

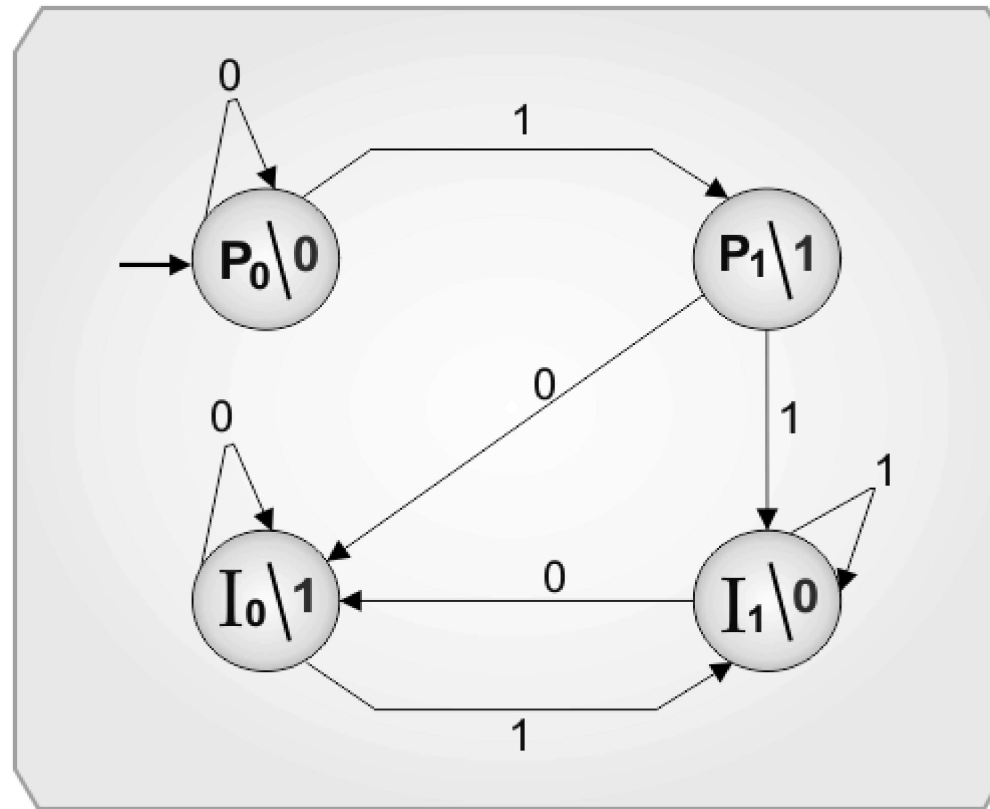
To find the 2's complement in a Moore machine, scan from LSB→MSB. Copy all bits up to and including the first 1 You see (this handles the "+1" by taking that 1 unchanged); after that first 1, output the bitwise complement (invert) of each remaining input bit. (Equivalent to: invert all bits, then add 1.)

The following table explains which states and outputs are used to construct the required Moore machine

State	Output	Meaning
P0	0	Pre-first-1 phase and this state will output 0 (used when the current input bit is 0)
P1	1	Pre-first-1 phase and this state will output 1 (used when the current input bit is 1) — this state is reached when we output the first 1 (the inclusive copy of the first 1) and on the next input we switch to invert-phase
I0	1	Invert (post-first-1) phase and this state outputs 1 (the inversion of input 0)
I1	0	Invert (post-first-1) phase and this state outputs 0 (the inversion of input 1)

(So P0/P1 = “still copying until first 1”; I0/I1 = “we are in invert-phase”. Each state’s output is the transformed bit for the current input that just caused the transition into that state.)

Moore Transition Diagram



2's Complement of Moore Machine

Explanation of transitions

- While in P_0 (haven't seen a 1 yet and current input was 0), if the next input is 0 we stay in P_0 and output 0 (copying 0). If next input is 1 we go to P_1 and output 1 (this is the first 1 we copy — it realizes the “+1”).
- From P_1 (we just emitted the first 1), on the next input we must switch to the invert phase: if next input is 0 $\rightarrow I_0$ (output 1), if next input is 1 $\rightarrow I_1$ (output 0).

- In the invert phase (I0/I1) we stay in invert-phase: the state chosen corresponds to the inverted output for the current input (so I0 outputs 1 when input is 0, I1 outputs 0 when input is 1), and transitions keep you inside I0/I1 appropriately.

Moore Transition Table

Present State	Input = 0	Input = 1	Output
P0	P0	P1	0
P1	I0	I1	1
I0	I0	I1	1
I1	I0	I1	0

Example Working

Take the binary number 1011 (MSB→LSB). We will feed bits LSB→MSB into the machine: that stream is 1, 1, 0, 1.

Start in P0.

Step	Input (LSB→MSB)	Transition (next state)	State Output (emitted bit)
1	1	P0 → P1	P1 outputs 1

Step	Input (LSB→MSB)	Transition (next state)	State Output (emitted bit)
2	1	P1 → I1	I1 outputs 0
3	0	I1 → I0	I0 outputs 1
4	1	I0 → I1	I1 outputs 0

Outputs (LSB→MSB) = 1 0 1 0. Rewriting MSB→LSB gives 0 1 0 1, which is 0101 — the correct two's complement of 1011.

Moore Machine Example 03: Moore Machine for Even Parity (running parity bit)

The machine outputs the running even-parity (0 = even number of 1s seen so far; 1 = odd number of 1s seen so far).

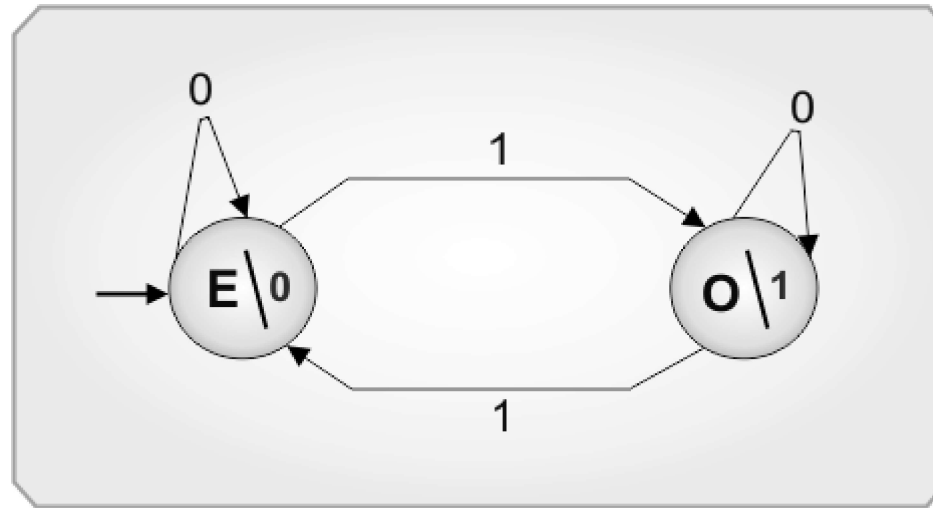
For each input bit, the machine updates its parity state and emits the parity after reading that bit.

Design of the Moore Machine

We use two states:

State	Output	Meaning
E	0	Even parity so far (emits 0)
O	1	Odd parity so far (emits 1)

Moore Transition Diagram



Moore Machine for Even Parity

Explanation

- Reading a “0” does not change parity, so the machine stays in the same state and emits that state’s output.
- Reading a “1” toggles parity: $E \rightarrow O$ (emit 1) or $O \rightarrow E$ (emit 0).

Moore Transition Table

Present State	Input = 0	Input = 1	Output
E	E	O	0
O	O	E	1

Moore Machine Example Execution

Let the input (MSB→LSB style) be 1 0 1 1. We feed bits left→right, and the machine starts in state E (even parity = 0).

Step	Input	Transition (next state)	State Output (emitted bit)
1	1	E → 1 → O	1
2	0	O → 0 → O	1
3	1	O → 1 → E	0
4	1	E → 1 → O	1

Final Output (for the input stream) = 1 1 0 1

(These are the running parity bits after each input — e.g., after reading the first 1 parity is odd → 1, after reading 1 0 parity remains odd → 1, etc.)

Moore Machine Example 04: Moore Machine for Detecting “11” Sequence

This Moore machine outputs 1 whenever the input sequence ends with two consecutive 1's, otherwise it outputs 0.

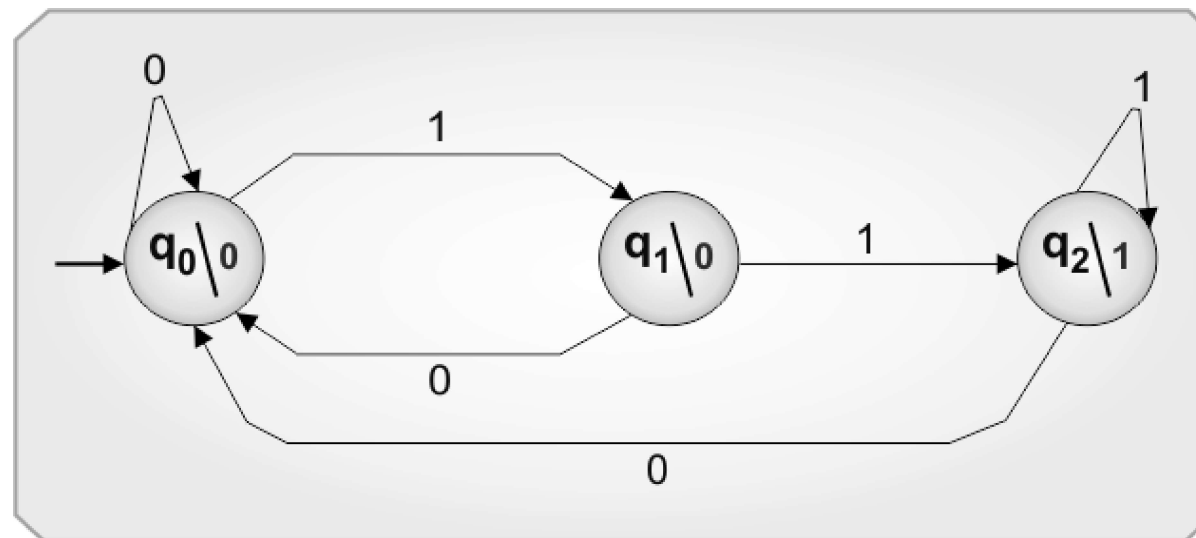
In other words, the output becomes 1 after seeing “11” at the end of the input sequence.

Design of Moore Machine

We use three states:

State	Output	Meaning
q0	0	Initial state — no relevant pattern seen yet
q1	0	The last input was 1, possible start of “11”
q2	1	“11” detected (the last two inputs were both 1’s)

Moore Transition Diagram



Moore Machine for Detecting “11” Sequence

Explanation

From q0:

- Input 0 → Stay in q0 (no pattern formed).
- Input 1 → Move to q1 (we've seen one 1).

From q1:

- Input 0 → Go back to q0 (pattern broken).
- Input 1 → Move to q2 (we've seen "11").

From q2:

- Input 0 → Go back to q0 (pattern broken).
- Input 1 → Stay in q2 (still ends in "11").

The machine's output depends only on the state — output 1 only when the last two bits were 1s.

Moore Transition Table

Present State	Input = 0	Input = 1	Output
q0	q0	q1	0
q1	q0	q2	0
q2	q0	q2	1

Moore Machine Example Execution

Let the input be: 1 0 1 1 1 0

Step	Input Bit	Transition (Next State)	State Output
1	1	$q_0 -1 \rightarrow q_1$	0
2	0	$q_1 -0 \rightarrow q_0$	0
3	1	$q_0 -1 \rightarrow q_1$	0
4	1	$q_1 -1 \rightarrow q_2$	1
5	1	$q_2 -1 \rightarrow q_2$	1
6	0	$q_2 -0 \rightarrow q_0$	0

Final Output: 0 0 0 1 1 0

This machine “remembers” the last two input bits and lights up (output = 1) whenever the input ends with “11”.